Minishell: A Small Shell with a Simulated Operating System

CmpSc 123 - Operating Systems

Group members:

Buduan, Tyron Clyde W.

Maligsay, Oryza M.

Orenza, Leah Joy D.

Sagsagat, Ronald Chris A.

November 2026

Language: Python 3

I. Overview of the Design

Minishell is a command-line program that behaves like a small Bash and also simulates two core parts of an operating system: CPU scheduling and deadlock handling. It is written in Python 3 using only the standard library. The library is used for regular expressions and basic file input/output is used for reading files. All scheduling and deadlock algorithms were written by our group from scratch.

The shell loop. The program shows the prompt minishell> , reads a line, splits it into tokens, runs it and prints the result, and repeats until the user types exit. Every step is wrapped so that a mistake by the user (bad syntax, missing file, unknown command) prints a clear message instead of crashing the shell.

The key design decision: one shape for every command. Every command is a function that takes (args, stdin_text) and returns (stdout_text, stderr_text, exit_code). An exit code of 0 means success. Because every command looks the same, the shell needs no special cases:

- a **pipe** (a | b) simply gives the stdout text of a to b as its stdin;
- **redirection** (>, >>, 2>, 2>&1) writes or reads that text from a file;
- && and || look at the exit code of the previous command;
- the two simulators, sched and deadlock, return the same triple, so sched rr p.txt > out.txt and deadlock detect s.txt | grep P1 works without extra code.

A line is processed in stages: text → tokens → split on ;, &&, || → split on | → pull out redirections → expand $VARIABLES → run the built-in command.

File	Purpose
minishell.py	Main loop, tokenizer, pipes, redirection, variables, if/then/else
builtins_cmds.py	File, directory, permission and variable commands
grep_cmd.py	Grep-like search
scheduler.py	PCB, six scheduling algorithms, sched command
deadlock.py	Detection, Banker's algorithm, deadlock command
compare.py	Module 4: runs all algorithm on the same data

II. Module 1: Shell Core

Built-ins. pwd, cd, ls (with -l and -a), mkdir, rmdir, touch, cp, mv, rm, cat, plus echo, wc, env, unset, test, true, false and help. ls -l uses os.lstat and stat.filemode to show

permissions such as -rwxr-xr-x. chmod accepts numeric modes (for example chmod 755 file) and rejects invalid modes with a usage message.

Tokenizer. We wrote our own tokenizer instead of splitting on spaces. It understands single quotes (nothing is expanded), double quotes (variables are expanded) backslash escapes, comments, and the operators ;, &&, ||, |, <, >, >>, 2>, 2>> and 2>&1. A plain 2 is only treated as a stderrr redirect when it is immediately followed by >, so echo 2 > f still works.

Redirection and pipes. >, >>, and < redirect stdout and stdin; 2> redirects stderr; 2>&1 merges stderr into stdout, which makes cat nofile 2>&1 | grep nofile work. Pipelines can have any number of commands.

Variables and conditions. NAME=value stores a variable, $NAME, ${NAME} and $? Read one, and env lists them .&& || and ; are evaluated left to right with Bash rules. As the bonus, a one-line if CMD; then CMD; else CMD; fi is supported (with test or [ ] as the condition. Nested if is not supported.

Grep-like search. Grep PATTERN [FILE] prints the line that contain the pattern and reads stdin when no file is given, so ls | grep txt works. The pattern is compiled with Python's re, so the bonus features (., [a-z], *, +, ?, backlash escapes and capturing groups) work too. Extra options are -i, -v, -n, -o, -c and -F (plain text). Its exit code is 0 for a match, 1 for no match and 2 for an error, and an invalid pattern gives a message rather than a crash.

Error handling. Unknown commands print "command not found (exit code 127)", missing files give the system's message with exit code 1, wrong arguments print a

usage line, and syntax errors such as echo hi > or ls || are reported. Even a bug inside a command is caught and shown as an error message.

III. Module 2: Process Scheduling Simulator

PCB and input. Each process has a Process Control Block holding its PID, arrival, burst time, priority, remaining time, state and completion time. Processes are loaded for a text file (PID ARRIVAL BURTS [PRIORITY]) or piped in on stdin. The reader reports the line number of bad input. The convention used is **lower priority number = higher priority.**

One shared output path. Each algorithm returns a timeline, a list of (pid, start, end) blocks, and sets each process' completion time. One shared function then prints everything: PCB table before and after the run, a text Gantt chart (with idle blocks when the CPU waits), the table of turnaround and waiting times, the averages, the number context switches and CPU utilization. The formulas are Turnaround = Completion - Arrival and Waiting = Turnaround - Burst. PCB states (New → Ready → Running → Ready/Terminated) are derived from the timeline, and --trace prints every state change. Every run uses a fresh deep copy of the process so one run never affects the next.

The six algorithms.

- **FCFS:** sort by arrival and run each process to completion; if the CPU is idle, jump the clock to the next arrival.
- **SJN(non-preemptive)**: whenever the CPU is free, choose the arrived process with the smallest burst.

- **Priority(non-preemptive)**: the same loop, but the key is the priority number.
- **Round-Robin:** a queue and a time quantum. A process runs for at most one quantum and then goes to the back; new arrivals join the queue before the process that was just preempted.
- **Multilevel Queue(MLQ):** Queue 1 (priority 1-2) uses Round-Robin with quantum 4; Queue 2 (priority 3 and above) uses FCFS. Queue 1 always runs first, and an arrival in Queue 1 pauses a running Queue 2 process. Processes never changes the queue.
- **Multilevel Feedback Queue (MLFQ):** Q0 is Round-Robin with quantum 2, Q1 is Round-Robin with quantum 4, Q2 is FCFS. Every process starts in Q0 and is demoted one level when it uses a full quantum without finishing. A new arrival preempts a process in Q1 or Q2, and the preempted process keeps its level.

Example Gantt chart produced by `sched rr --quantum 4 samples/processes.txt`:

```
|  P1 |  P2 |  P3 |  P4 |  P1 |  P3 |  P4 |  P3 |
0     4     8     12    16    20    24    25    26
```

IV. Module 3: Deadlock Simulator

Input. The file gives the number of processes and resource types, then the allocation matrix, the Max matrix, an optional Request matrix and the Available vector. Need = Max - Allocation. The reader checks the sizes and rejects negative numbers or a process holding more than its Max.

Detection. The program starts with Work = Available and treats a process that holds nothing as already finished. It then repeatedly finds an unfinished process whose Request (or Needm if no Request matrix is given) is at most Work, lets it finish and adds

its Allocation back to Work. Any process that can never finish is deadlocked. For a deadlocked system the bonus recovery suggestion repeatedly terminates the stuck process holding the most resources and re-runs detection until the deadlock clears.

Banker's safety check. The same loop uses Need:a process can finish if Need< Work, and then it returns its Allocation. If every process can finish the state is safe, and the order in which they finished is the safe sequence. The program prints each step so the reasoning can be followed.

Resource requests. For a request by process i:(1) if the request exceeds Need, it is as an error;(2) if it exceeds Available, the process must wait; (3) otherwise the request is pretend-granted on a copy of the state; (4) the safety runs on the new state, and the request is granted if the state is safe of denied if not. The real state is never modified.

Results on the textbook example(samples/sysrem_safe.txt):

Test	Result
Safety check	SAFE, safe sequence P1 → P3 → P4 → P0 → P2
P1 requests (1, 0, 2)	GRANTED (the new state is still safe)
P4 requests (3, 3, 0)	DENIED (would leave an unsafe state)
P0 requests (4, 0, 0)	Must WAIT (more than Available)

P1 requests (2, 0, 0)	ERROR (more than its Need)

Unsafe example (samples/system_unsafe.txt): P0, P1 and P2 each hold one resource and each needs the one held by the next process, a circular wait with nothing available. P3 holds nothing and finishes immediately. Detection reports **DEADLOCK: P0, P1, P2**, and the recovery suggestion shows that terminating P0 releases its resource and clears the deadlock.

V. Module 4: Comparison Report

We ran four workloads through all six algorithms using compare.py. All numbers below were produced by our own simulators (Round-Robin quantum 4; MLFQ quantum 2/4; MLQ Queue 1 = priority 1-2). Context switches count changes of the running process; the cost of a switch is not added to the waiting time.

Workload A: staggered arrivals (P1 0/8, P2 1/4, P3 2/9, P4 3/5, written as arrival/burst)

Algorithm	Avg Waiting	Avg Turnaround	Max Waiting	Context Switches
FCFS	8.75	15.25	18	3
SJN	7.75	14.25	15	3
Priority	7.75	14.25	15	3

Algorithm	Avg Waiting	Avg Turnaround	Max Waiting	Context Switches
Round-Robin (q=4)	11.75	18.25	17	7
MLQ (q=4)	8.75	15.25	15	5
MLFQ (q=2)	13.00	19.50	15	9

Workload B: all five processes arrive at time 0 (bursts 10, 1, 2, 1, 5)

Algorithm	Avg Waiting	Avg Turnaround	Max Waiting	Context Switches
FCFS	9.60	13.40	14	4
SJN	3.20	7.00	9	4
Priority	8.20	12.00	18	4
Round-Robin (q=4)	7.40	11.20	12	7
MLQ (q=4)	8.20	12.00	18	4
MLFQ (q=2)	5.80	9.60	10	7

Workload C: mixed workload with one long job (8 processes; P1 arrives first with burst 24)

Algorithm	Avg Waiting	Avg Turnaround	Max Waiting	Context Switches
FCFS	27.25	34.50	41	7
SJN	22.75	30.00	38	7
Priority	25.00	32.25	41	7
Round-Robin (q=4)	18.00	25.25	34	15
MLQ (q=4)	14.25	21.50	40	9
MLFQ (q=2)	19.75	27.00	41	16

Workload D: idle gap (P1 0/3, P2 10/4, P3 11/2). Every algorithm gives an average waiting time of 1.00 and turnaround of 4.00, because there is almost no competition for the CPU (MLFQ differs only in who waits). This workload mainly tests that the idle gap is handled correctly.

Mean of the four workloads

Algorithm	Mean Avg Waiting	Mean Avg Turnaround
MLQ	8.05	13.19

Algorithm	Mean Avg Waiting	Mean Avg Turnaround
SJN	8.68	13.81
Round-Robin	9.54	14.68
MLFQ	9.89	15.03
Priority	10.49	15.62
FCFS	11.65	16.79

Analysis and Conclusions

Which algorithm performed best, and why. SJN is the most dependable. It produced the lowest average waiting time on Workload A (tied with Priority) and Workload B (3.20 vs. 9.60 for FCFS), aligning with the theory that executing shortest jobs first minimizes wait times. It also ranked second overall across all workloads (8.68 mean). Although MLQ had a slightly lower overall mean (8.05), its success depended entirely on Workload C, where short jobs happened to receive high priorities(1-2). On Workloads A and B, MLQ performed no better than FCFS or Priority (8.75 and 8.20), making SJN the most dependable choice.

Where our data disagrees with the usual claim that SJN is best. On workload C, SJN (22.75) was clearly beaten by Round-Robin (18.00) and MLQ (14.25). The reason is that our SJN is non-preemptive and the 24-unit process P1 arrives first, so it starts at

time 0 and everything else must wait for it. Round-Robin and MLQ can interrupt a long job. So SJN is only best when long jobs do not get a head start.

Fairness and starvation. Round-Robin had the lowest longest wait on workload C (34, against 38 to 41 for the others) and a low one on B (12), which fits its purpose of sharing the CPU fairly. SJN's well–known weakness, starvation of long jobs, cannot show up in our data, because every process list is finite and all jobs eventually run. It would only appear if short jobs kept arriving forever.

The price of Round-Robin and MLFQ. Round-Robin made 7, 7 and 15 context switches on workloads A, B and C, against 3, 4 and 7 for FCFS, and MLFQ made 9, 7 and 16. Our averages ignore the cost of a switch, so on a real machine these two would look worse than shown. On workload A, where all processes have similar lengths (4 to 9) and the quantum is 4, Round-Robin(11.75) was worse than FCFS(8.75), because slicing the jobs only delays when each one finishes.

MLFQ. was never the best, but on workload B it came second (5.80) without knowing the burst lengths in advance. Demoting jobs that use many quanta makes it behave somewhat like SJN, which is the main reason to use it. On workload A it was the worst (13.00), because of any short slices at quantum 2.

Conclusion. No single algorithm wins everywhere. With known burst lengths and no long job blocking the start, use SJN. When one long job arrives early and

responsiveness matters, a preemptive scheme (Round-Robin, MLQ or MLFQ) is better. FCFS was the worst on average (11.65) in every workload where processes comppete for CPU.

Deadlock examples. The safe example gives the safe sequence P1 → P3 → P4 → P0 → P2. The unsafe example has no safe sequence and detection fins P0, P1 and P2 deadlocked (see Section 4).

VI. Testing

Running python3 -m unittest discover -s tests executes 68 tests, all passing. The expected scheduling values were calculated by hand first (for example FCFS on workload A: completions 8, 12, 21 and 26 give waits of 0, 7, 10 and 18, average 8.75). The Banker's tests use the well-known textbook example. Besides the expected numbers, the scheduler tests also check that no algorithm runs two things at once, runs a process before it arrives, or gives a process more or less CPU time than its burst.

Area	Test 1	Test 2	Edge cases
Scheduler	all arrive at time 0	staggered arrivals; mixed workload	single process, empty file, idle gap, bad input file

Area	Test 1	Test 2	Edge cases
Deadlock	safe state with sequence	unsafe/deadlocked state	one process, nothing allocated, zero processes, malformed file
Shell	pipes and redirects	&&, ||, ;, variables, if/else	unknown command, missing files, syntax errors, exit

VII. Simplifications and Limitations

- Pipe data is passed between commands as text in memory, not through real operating-system pipes. This is acceptable for a simulation.
- Only built-in commands are available; external programs are not launched, so unknown names print “command not found”.
- if/then/else must be on one line and cannot be nested. Background jobs (&) are not supported.
- chmod supports numeric modes only.
- The scheduler assumes processes never block on I/O, so the Waiting PCB state is not used, and context-switch cost is zero.

VIII. Member Contribution

Member 1 (Shell Lead): Ronald Chris A. Sagsagat.

- Wrote the main loop and the tokenizers, which splits a typed line into words and operators and handle quotes.
- Built pipes and the redirections (>, >>, <, 2>, 2>>, 2>&1).
- Added variables ($Name, $?) and the operators &&, || and ;, plus the if/then/ else bonus.
- Wrote the built-in commands (ls -l, cd, mkdir, cp, chmod, and others).
- Built the error handling that keeps the shell from crashing.
- Integrated grep, sched and deadlock into the shell, so their output can be piped and redirected.

Member 2 (Scheduler Lead): Leah Joy D. Orenza.

- Wrote the PCB and the process file reader, which reports the line number of bad input.
- Implemented the three algorithms FCFS, SJN, and Priority
- Implemented the three algorithms: Round-Robin, Multilevel Queue, and Multilevel Feedback Queue.
- Built the shared output: text Gantt chart (including idle time), waiting and turnaround table, averages, and PCB state table with –trace.
- Wrote the sched command with the –quantum option.

Member 3 (Resources Lead): Oryza Maize M. Maligsay.

- Wrote the system file reader with input checks.
- Implemented deadlock detection.
- Implemented the Banker's safety check, which prints each step and the safe sequence.
- Built request handling (granted, denied, must wait, or error) and the recovery suggestion bonus.
- Wrote the deadlock command and tested it with the textbook example.

Member 4 (Docs and Testing Lead): Tyrone Clyde W. Buduan.

- Wrote the grep command, including the regular-expression bonus.
- Created the sample input files and the 68 tests, including edge cases (empty queue, single process, idle CPU, bad files).
- Built compare.py for the Module 4 comparison.